

cidoc-crm-mcp

An introduction for people who model in CIDOC CRM and have never had to care what an MCP server is.

NO PRIOR SOFTWARE KNOWLEDGE ASSUMED · COMPANION TO docs/how-it-works.html

1 What it is

Two things become practical when an assistant can look the CRM up instead of recalling it. The larger one is converting a collection — not one object, all of them.

Ask a chat assistant a CIDOC CRM question today and the answer comes back immediately and fluently. It names a class, and the class is plausible. Sometimes it is right; sometimes the property it suggests has a domain that forbids the very thing you asked about; sometimes it names a class the standard retired years ago. Nothing in the answer tells you which of those you have.

The problem is not carelessness. It is that nothing *authorises* the answer — the assistant is recalling the CRM the way you might recall a scope note you last read in 2019. This gives it somewhere to look instead: twelve tools, each wired to a source you can go and read.

Converting a collection

A museum catalogue, an excavation archive, a card index typed up in 1994, a shelf of wiki articles. Somebody has to decide that this field is an E22, that this date is an E52 Time-Span reached through P4, that the maker is an E21 joined to the object by an E12 Production — and then do it for every record in the collection.

The deciding was always expert work. The *doing* is what turned a task into a project, and the bottleneck was never writing the RDF. It was knowing whether what you had written was right, with no way to find out except an expert reading it, record by record.

That is the part that changes. An assistant with these tools settles the mapping against the specification rather than from memory, writes the RDF with the exact identifiers the standard uses, and then submits the whole export to a validator that answers for **every** triple: legal, illegal, or a name that is not a CRM property at all. Not a sample, and not only the links somebody thought to doubt. Measured on this build: a 10,000-object export — 30,000 links — checked in **eight seconds**.

Two things make that trustworthy rather than merely fast.

The first is that the validator reads the file, and not the assistant's account of the file. That sounds like a quibble and is not. An assistant asked to check its own work will start by extracting what it believes it wrote, and the extraction is where an error quietly disappears. One run pulled a hundred shapes out of its own finished XML, resolving each name to an identifier on the way — so `changed_ownership_by`, which is not a CRM label at all, reached the validator already turned into P24. It passed, and shipped four times. An earlier attempt with no validator, checking the names

by hand against real labels, had caught that same string. Having a validator had made the checking narrower, and reading the artifact is what fixes that.

The second is that every finding names the record it came from, so a report on a hundred thousand objects is a worklist rather than a verdict.

Asking the model questions

The everyday use, and the one that keeps interrupting the other. What does this class actually mean, and how does it differ from the one beside it? Which property can legally join these two? Is the thing I just wrote legal CRM, and if not, which part of it is not? These are answered by computing over the standard's own machine-readable definition, so they come back the same every time, with no ranking and nothing nearly included.

And, underneath both, the reasoning

A scope note says what a class is. It rarely says what it is *for*, which of two plausible classes the committee actually intends, or what was tried and rejected on the way to the current wording. That reasoning exists, in twenty-six years of CRM-SIG mailing list argument, and it is here too — so when a mapping stalls on a question the specification does not settle, the answer can be quoted with a name and a date attached rather than invented. It is the part of this that has no substitute anywhere else.

What an MCP server is

MCP stands for **Model Context Protocol**. It is an agreed way for an AI assistant to call out to a tool and get a real answer back, rather than recalling one. An **MCP server** is a program that offers such tools. The name is unhelpful in one specific way: it is not a website and there is nothing to log into. Think of it as a reference desk that only your assistant talks to.

You connect it once — one line of configuration in whatever assistant you use, shown in section 6 — and then you never think about it again. You ask your questions the way you always did. The difference is what happens in the seconds before you get an answer.

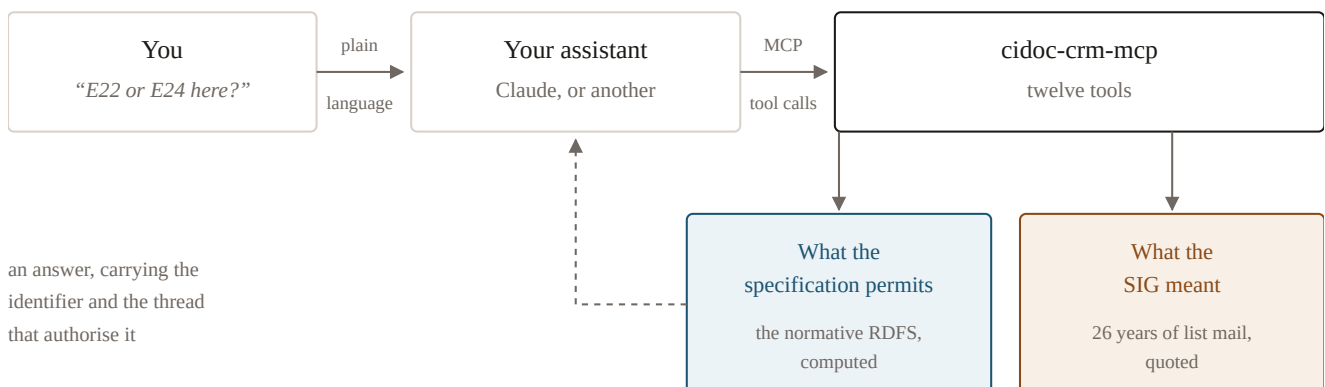


Figure 1. Where you sit. You never address the boxes on the right; you talk to the second box from the left, exactly as before. What you gain is not a new application to learn but a different kind of answer — one that arrives carrying the identifier and the thread that authorise it.

What a session actually looks like

You are cataloguing a photograph of a building. You ask your assistant how to model it. Before answering, it makes perhaps four calls you never see: it searches the specification for the relevant guidance, looks up the two classes it is choosing between, asks which properties can legally join them, and — once it has written some RDF — asks whether that RDF is legal. Then it answers you, and the answer says *E36 Visual Item*, and cites the section it came from, and you can go and read that section.

If your question is instead *why* the CRM is shaped this way — why production is an event rather than a property, what happened to E84 — the assistant reaches into the other half: the mailing list, where that was argued out, often at length, sometimes twice.

A conversion looks different only in scale. You hand over a batch of records, agree a mapping on the first few, and then the work becomes mechanical: for each record the assistant writes the RDF and submits the whole document to the validator, which answers for every triple rather than for the ones anybody thought to doubt. What comes back to you is the conversion plus a list of what did not pass — which is the part you actually have to read.

2 The two halves, and why they are kept apart

One half computes what the standard permits. The other quotes what people said. Blending them would produce a more confident answer and a worse one.

The twelve tools are not one set. They divide down the middle, and the division is the single design decision everything else follows from.

THE NORMATIVE HALF – SIX TOOLS

Answers computed from the CIDOC CRM's own machine-readable definition. When this half says *P108 has produced runs from E12 Production to E24 Physical Human-Made Thing*, that is not an opinion and not a recollection. It is read off the standard. When it says a link is illegal, the link is illegal.

THE EVIDENTIARY HALF – SIX TOOLS

Answers quoted from twenty-six years of CRM-SIG mailing list traffic, the issue register, and the meeting minutes. This half does not tell you what is correct. It tells you *who said what, and when* — including people who turned out to be wrong, which is exactly why the attribution matters.

The temptation is to merge the two into a single confident answer. The tools refuse to. A claim from the normative half and a claim from the archive have different standing, and a reader who cannot tell them apart has lost the only thing that made the archive worth consulting.

The twelve

`crm_concept`

One identifier or label: definition, hierarchy, domain and range, its siblings, and how often the archive argued about it.

`crm_list`

Every identifier in one model, with the exact spelling to write in RDF.

`crm_connect`

Every property that can legally join two classes, both directions.

`crm_validate_link`

Is this one subject–property–object triple legal?

`crm_validate_rdf`

Check a whole model in Turtle, RDF/XML, N-Triples or JSON-LD.

`crm_validate_xml`

Check a whole document in the published CIDOC CRM example format.

`crm_search`

Search the mailing list.

`crm_docs`

Search the specification itself, the issue register, the minutes, or the Conceptual Modelling Principles.

`crm_issue`

One SIG issue by number: its status, its outcome, and every thread and meeting that discussed it.

`crm_thread`

Read a whole discussion in date order.

`crm_show`

Read one message, or one section of the specification, in full.

`crm_quote`

Verify that a quotation actually occurs in the source it is attributed to.

That last one deserves a sentence. An assistant that has read a long thread can reconstruct a quotation from memory and get it subtly wrong — and a human reviewer, reading the same thread, can be talked into confirming it. `crm_quote` decides the question mechanically instead: the phrase is in the message, or it is not.

3 What is actually inside

The software is the small part. The corpus is what makes the answers possible, and it is worth

knowing its shape before you rely on it.

THE NORMATIVE HALF	COUNT
CRMbase classes	83
CRMbase properties	168
Properties of properties (P14.1 and its kind)	16
Family-extension identifiers, across 14 models	610
Historical identifiers, no longer defined	59

Table 1. The normative half, from the CIDOC CRM v7.1.3 RDFS together with the published family-extension encodings. *Historical* identifiers are ones the archive discusses that the current standard no longer defines — kept deliberately, because the useful fact about E84 is precisely that it is dead.

THE EVIDENTIARY HALF	COUNT
CRM-SIG messages, 21 Aug 1999 – 15 Jan 2026	5,400
Reconstructed threads	1,878
Summarised episodes	883
SIG issues with discussion or recorded outcome	716
Specification declarations (one per class or property)	244
Narrative guidance sections	130
Conceptual Modelling Principles sections	66
Issue-register page sections	3,002
SIG meeting-minute sections	3,644

Table 2. The evidentiary half. A *thread* is a reconstructed conversation; an *episode* is a topically coherent run of messages within one, summarised so that a debate which trails off and resumes two years later can still be found as one thing.

Two things follow from these numbers that are easy to miss.

First, **the specification is a small minority of the searchable text.** The 244 declarations and 130 narrative sections sit in the same store as 6,646 sections of issue pages and minutes. Left to compete on size alone, the specification loses every time — so a search of the reference documents returns the specification by default, and the issue register and minutes only when asked for by name.

Second, **the archive is not a neutral sample of opinion.** It is the working correspondence of a standards committee. Some contributors post constantly; some questions were settled in one exchange and some were reopened for a decade. The volume of discussion about an identifier measures how contentious it was, not how important it is.

4 Two kinds of search, and why one is not enough

Only two of the twelve tools search at all. Those two search everything twice, by methods that fail in opposite directions, and combine the results.

It is worth being exact about which half this concerns, because it is the sharpest difference between the two. **The normative half does not search.** When `crm_connect` tells you which properties join an E21 to an E53, it has not looked for likely-seeming ones: it has walked the class hierarchy and tested every property's declared domain and range. The answer is computed, and it is the same answer every time. There is no ranking, nothing was nearly included, and no passage had to be found.

Searching belongs to the other two tools — `crm_search` over the mailing list, and `crm_docs` over the reference documents. (The second of those searches the specification's own text as well as the issue register and the minutes; it is retrieval either way, which is what puts it on this side of the line.) What follows is about those two.

Suppose you want the discussion about controlled vocabularies. There are two ways a computer can find it, and each has a blind spot the other covers.

Keyword search, which is literal

The first method indexes the words as written and finds documents containing them. It is exact and fast, and it is the only thing on earth that can reliably find `E55` — a short token, meaningless as English, that has to match precisely. Its blind spot is that it has no idea `E55` and the phrase *controlled vocabulary* have anything to do with one another. Ask it for one and it will not show you the other.

Meaning-based search, which is associative

The second method converts each passage into a long list of numbers — an *embedding* — positioned so that passages about similar things land near one another, whatever words they used. It will find a message about controlled vocabularies that never once writes that phrase. Its blind spot is the mirror image: identifiers are gibberish to it. The model that produces these numbers does not know `E55` as a word at all; it breaks it into a meaningless `E` and a `55`, and cheerfully lands it next to E52 and E41, which are unrelated.

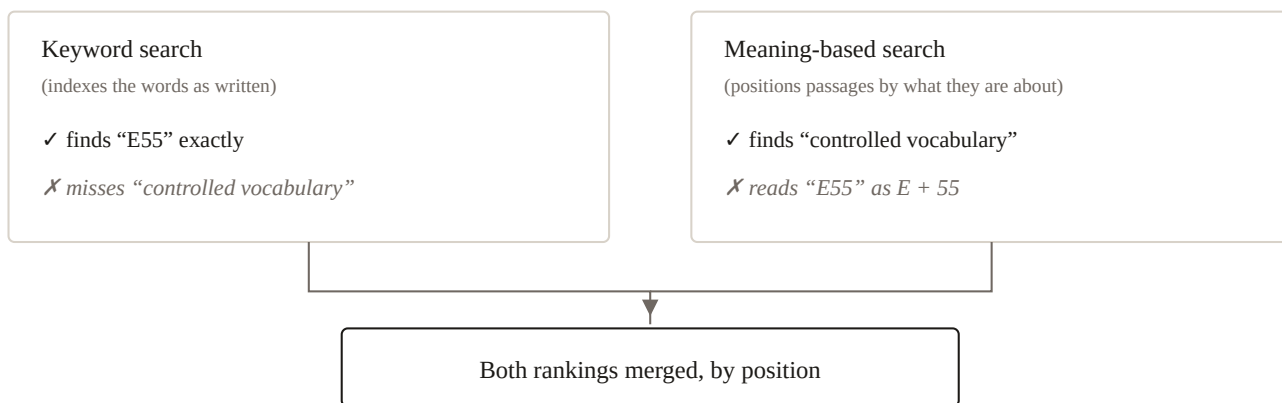


Figure 2. The two methods fail in opposite directions, so every search runs both. They are combined by **where** each result ranked rather than by **how strongly** each method scored it — the two scores are measured in incompatible units and cannot be meaningfully averaged. Comparing positions instead means there is no weighting to tune, and no number anyone had to pick.

This is why, when you search, an identifier and a paraphrase both work, and why you do not have to know which kind of question you are asking. It is also why a search result and a validator's verdict should not be read the same way. One is the best of what was found; the other is what the standard says.

Where the material came from

The normative half is built from the CIDOC CRM's own published files — the v7.1.3 RDFS, the family-extension encodings, and the v7.3.2 text for the first-order logic and full-path expansions the older files do not carry. The evidentiary half is built from the public CRM-SIG list archive, the issue register pages, and the published meeting minutes.

Nothing was paraphrased on the way in. What the tools quote is what the sources say, and every quotation carries the identifier or thread it came from so you can go and check it. One step did use a language model: the 883 episode summaries, which condense a thread into what was asked, who argued what, and whether it was settled. Those are summaries and should be read as such — the underlying messages are always one call away, and the summaries exist to help you find them, not to replace them.

5 What not to trust it for

A tool that tells you its limits is more useful than one that does not, and these limits are specific.

The archive records arguments, not conclusions

This is the big one. A search result is a thing somebody said on a mailing list. It may be a considered position from the CRM's principal author; it may be a first reaction that the next message demolished. The tools deliberately do not adjudicate. They give you the author and the date and the thread, and reading on is your job. Where a question was settled *formally* it was settled as a numbered issue — `crm_issue` is the tool that reaches the decision record rather than the debate.

The specification is domain-agnostic, and so are the answers

The CRM does not mention grave goods, ship burials or daguerreotypes, and it never will — enumerating concrete cases is precisely what its Minimality principle forbids. A subject-matter search of the specification therefore finds nothing, and that absence is itself the answer: model it from the generic classes. Mapping your actual object onto E36 or P62 is the modeller's job, and the assistant's. It is not something the specification will do for you.

"Not checked" is not the same as "fine"

When the validator reports a predicate as *not CRM*, or a subject as *unchecked*, it means exactly that: outside this validator's authority, or never examined, because the thing had no declared type to check against. Neither is an endorsement. This matters most for the commonest error an assistant makes, which is not an illegal link but a plausible-looking class name the standard renamed years ago.

The corpus has a date, and this document may not match it

Every answer is drawn from one build of the corpus, whose contents are fixed at the moment it was made. Discussion after that date is not in it. The counts in section 3 describe the build current when this was written; a running server reports its own provenance:

```
docker run --rm ghcr.io/stefdoerr/cidoc-crm-mcp:latest cat /app/corpus-provenance.json
```

There is no authentication

The server itself has no logins and no access control, by design — a copy running on your own machine is reachable only from it, and that is the whole protection. Anyone who can reach the port can search the whole archive. So a copy put anywhere reachable needs a gate in front of it: the hosted instance in section 6 sits behind a proxy that checks a token and refuses everything else, which is what makes an address on the open internet safe to hand out.

6 Trying it

Two routes: use the hosted instance with a token, or run the whole thing yourself. Either way it is one line of configuration and there is nothing to install.

The hosted instance

If you have been sent an access token, everything is already running and you need nothing on your own machine. In Claude Code it is one line — put the token from your invitation in place of the placeholder:

```
claude mcp add --transport http cidoc-crm https://cidoc.stefdoerr.com/mcp \
--header "Authorization: Bearer YOUR_TOKEN_HERE"
```

Assistants that take a configuration file want the same three things — the address, the transport, and the token:

```
{
  "mcpServers": {
    "cidoc-crm": {
      "type": "http",
      "url": "https://cidoc.stefdoerr.com/mcp",
      "headers": { "Authorization": "Bearer YOUR_TOKEN_HERE" }
    }
  }
}
```

The token is what the server checks before it answers anything, and it is shared by everyone testing: treat it like a door key rather than a password. Keep it out of anything public — a repository, an issue, a screenshot, a pasted config in a chat. Without it every request is refused.

Or run your own

Everything — the code, the ontology and the whole corpus — ships in a single container image, for Intel and Apple Silicon alike. Start it:

```
docker run -d -p 127.0.0.1:8000:8000 ghcr.io/stefdoerr/cidoc-crm-mcp:latest
claude mcp add --transport http cidoc-crm http://127.0.0.1:8000/mcp
```

No token here, and none needed: the 127.0.0.1 is doing real work, publishing the port to your own machine only. That is the arrangement section 5 describes, and the reason your own copy needs no key is that nothing else can reach it.

Ask your assistant a CRM question afterwards and you should see it consult the tools before it answers.

The advice it gives is a file you can argue with

The server ships a modelling brief — the instructions an assistant reads before working through a model with you. It is advice, it was wrong in ways only real modelling runs revealed, and it will be wrong again. So it is a file rather than something buried in the program. Mount your own directory over it and your version wins, re-read on every request, with no rebuild and no restart:

```
docker run -d -p 127.0.0.1:8000:8000 \
-v "$PWD/prompts:/prompts:ro" ghcr.io/stefdoerr/cidoc-crm-mcp:latest
```

If you disagree with how it tells an assistant to approach a model — and if you model in CRM seriously, you will — that is the file to change.

Where to go next

[docs/how-it-works.html](#) is the companion to this document: every one of the twelve tools with a real call and the output that call actually returned, one question worked through end to end, and

the details of where each part of the corpus came from. Open it in a browser from a clone — GitHub shows HTML as source rather than rendering it.

`cidoc-crm-mcp` · github.com/stefdoerr/cidoc-crm-mcp

Counts in section 3 are from the corpus build current at the time of writing; a running server reports its own, as shown in section 5.